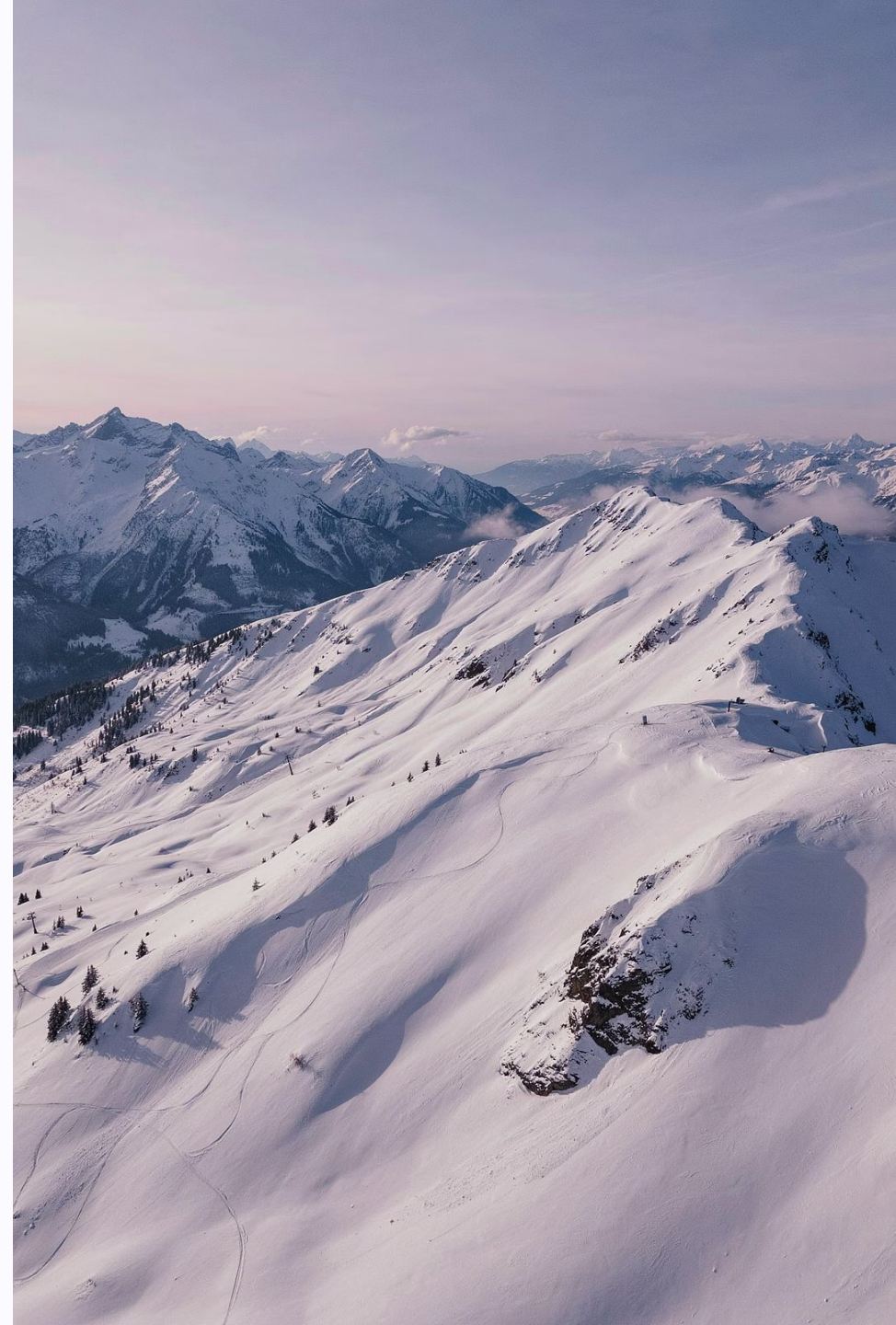


MODELADO ORIENTADO A OBJETOS

Restricciones {XOR} y superclase deducida

Cuando el enunciado te dice más de lo que parece

Cómo identificar jerarquías implícitas y restricciones de exclusión mutua en el dominio del problema.



Superclase deducida: cuando el enunciado no la nombra

A veces el enunciado **nunca dice explícitamente** "esto es un tipo de eso". No aparece la palabra superclase, no hay flecha de herencia dibujada, no hay una clase padre mencionada.

Sin embargo, dos clases distintas pueden participar de las **mismas relaciones** con otras partes del sistema: compiten en las mismas pruebas, se inscriben en los mismos torneos, tienen resultados del mismo tipo.

Cuando eso ocurre, el modelador **deduce** una superclase común aunque el enunciado nunca la haya nombrado con esa palabra. La superclase no la inventamos: la inferimos del comportamiento compartido que el enunciado sí describe.

¿Cuándo se deduce?

- Dos clases participan de las mismas relaciones
- Comparten el mismo rol frente a una tercera clase
- Tienen resultados o atributos del mismo tipo
- El enunciado las trata como "la misma clase de cosa"
- El dominio las agrupa conceptualmente

Campeonato de Esquí: deduciendo *Participante*

Leemos el enunciado: "**Los esquiadores pueden competir individualmente o formando parte de un equipo, pero nunca de las dos formas a la vez.**"

Lo que el enunciado dice

Hay **Esquiador** y hay **Equipo**. Ambos compiten en pruebas. Ambos tienen resultados. El enunciado los trata como participantes de la misma actividad.

Lo que el modelador infiere

Si Esquiador y Equipo comparten el mismo rol frente a Prueba, entonces existe una abstracción común: algo que *participa*. Esa abstracción es **Participante**.

La superclase resultante

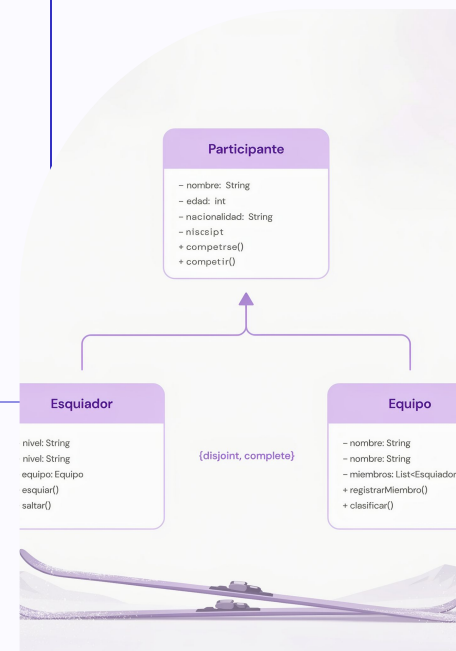
Participante {disjoint, complete}: toda instancia es Esquiador o Equipo, nunca ambas cosas, y no puede ser un Participante genérico sin ser una u otra.

Participante

Superclase común

Equipo

Compite en grupo



Esquiador

Compite individualmente

¿Por qué mantener *Participante* si Python no lo exige?

En Java: el compilador lo exige

Si querés meter un **Esquiador** y un **Equipo** en la misma lista y tratarlos de forma uniforme, el compilador exige un tipo común. Sin una superclase compartida, no hay polimorfismo posible — el código directamente no compila.

La jerarquía en Java es, en parte, una imposición técnica del lenguaje de tipado estático.

En Python: duck typing

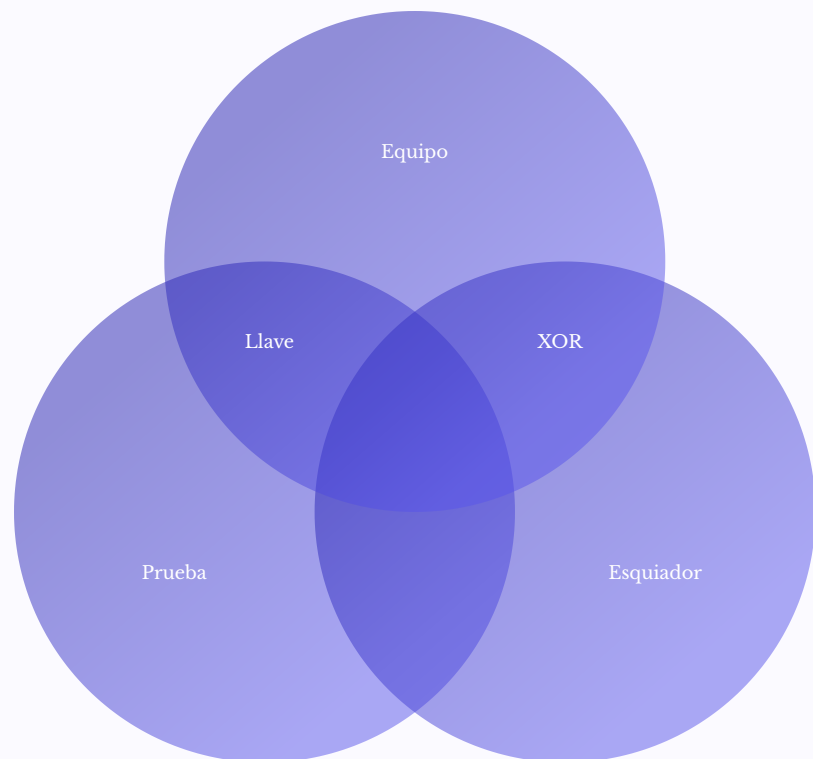
Si el objeto tiene el método, alcanza. No importa de qué hereda. Python no necesita que Esquiador y Equipo compartan una superclase para que el código funcione polimórficamente.

Entonces, ¿si Python no lo exige, para qué está **Participante** en el modelo?

La respuesta clave 🎯

Participante no está ahí por el compilador. Está porque **el enunciado dice que Esquiador y Equipo son la misma clase de cosa**. Que Python no te obligue a esa jerarquía y la mantengamos igual es la prueba de que es una decisión de diseño real — una verdad del dominio — y no una imposición del lenguaje.

La restricción {XOR}: exclusión mutua entre relaciones



La restricción **{XOR}** marca que dos relaciones son **mutuamente excluyentes** para el mismo objeto. No es una restricción sobre atributos: es una restricción sobre la participación en relaciones.

¿Qué significa {XOR}?

Un mismo objeto puede participar en una relación **o** en la otra, pero nunca en ambas al mismo tiempo. Es un "o exclusivo" estricto a nivel de modelo.

En el ejemplo del esquí

"Nunca de las dos formas a la vez" es exactamente una restricción {XOR}: un esquiador no puede estar simultáneamente en **modo individual** y en **modo equipo**.

¿Dónde se marca?

En el diagrama, la marca {XOR} aparece sobre las dos relaciones afectadas, indicando al lector del modelo que la combinación simultánea es inválida por definición del dominio.

La superclase resuelve un problema concreto: correlación de tipos

Más allá de reflejar el dominio, tener **Participante** como superclase común resuelve un problema real en el código: cómo manejar "cualquiera de los dos" sin preguntar el tipo en cada lugar.

01

Sin superclase común

Para guardar el vencedor de una prueba individual necesito una referencia a **Esquiador**. Para el vencedor de una prueba por equipos, una referencia a **Equipo**. Son dos campos distintos, dos lógicas distintas, y **if/else en todos lados**.

02

Con Participante como superclase

El vencedor de cualquier prueba es simplemente un **Participante**. El código no necesita saber si es Esquiador o Equipo: trabaja con la abstracción común y polimorfismo hace el resto.

03

El resultado en el modelo

Una prueba tiene exactamente un **Participante** vencedor. Punto. La superclase deducida convierte una complejidad condicional en una relación simple y limpia.



Regla de oro: Cuando el mismo atributo o relación en una tercera clase necesita apuntar a dos tipos distintos, eso es una señal fuerte de que esos dos tipos comparten una superclase.

Dos confusiones frecuentes — y cómo evitarlas

Confusión 1: {XOR} sobre atributos

{XOR} **no** significa que el objeto no puede tener dos atributos con el mismo valor a la vez. Eso sería una validación de datos.

{XOR} significa que el objeto **no puede participar simultáneamente** de dos relaciones o roles concretos definidos en el modelo. Es una restricción estructural, no de datos.

Confusión 2: superclase solo para el compilador

La superclase deducida **no** aparece porque el lenguaje lo exija. Aparece porque dos clases comparten el mismo tipo de relación con una tercera clase — eso es una verdad del dominio.

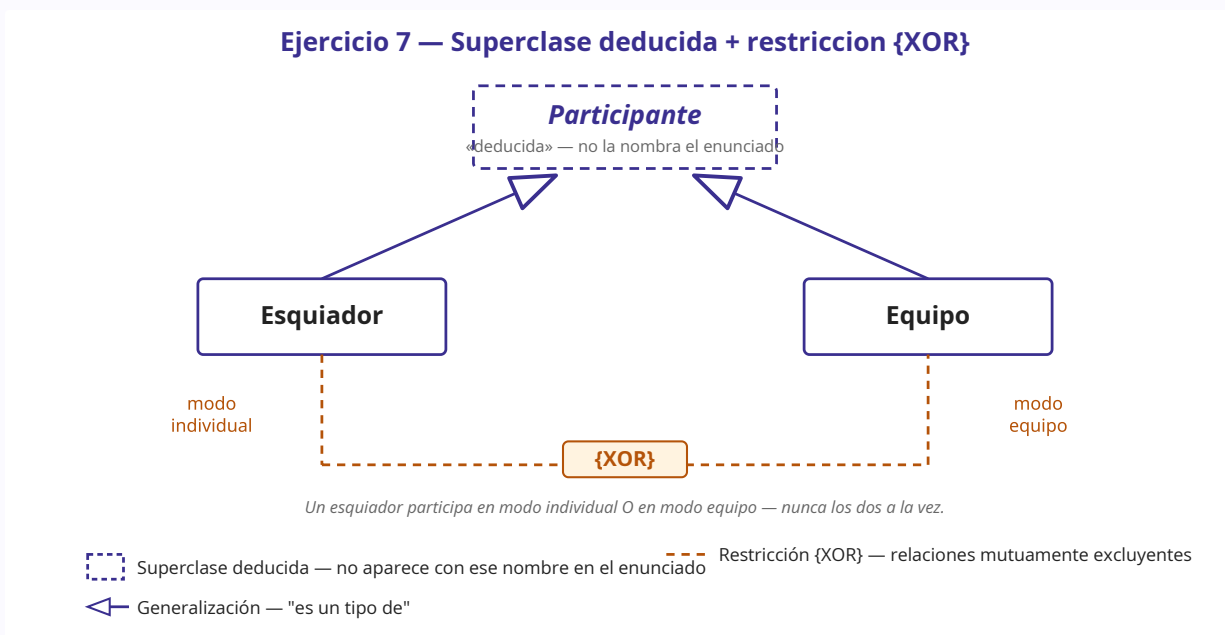
En Python, que el lenguaje no la exija y la mantengamos igual confirma que la decisión viene del **dominio**, no de la sintaxis.

La señal para detectar una superclase deducida

Buscá dos clases que aparezcan en el mismo rol frente a una tercera. Si el enunciado las trata como "la misma clase de cosa" aunque no lo diga con esas palabras, ahí está tu superclase — solo hay que nombrarla y hacerla explícita en el modelo.

El modelo completo de un vistazo

Todo lo que vimos en este video, condensado en una sola estructura. La superclase deducida en el centro; la restricción {XOR} marcando la exclusión mutua entre los dos modos de competir.



Superclase deducida

No siempre está escrita en el enunciado. Aparece cuando dos clases comparten el mismo rol frente a una tercera. Hay que **inferirla**.

Restricción {XOR}

Exclusión mutua entre **relaciones**, no entre atributos. Un objeto no puede participar simultáneamente de los dos roles marcados.

Python vs. dominio

Que Python no exija la jerarquía confirma que mantenerla es una decisión del **dominio**, no del lenguaje. Eso es diseño real.